

7. Bezpieczeństwo sieci – homework

(README.md)

Uruchamiam skrypt: "start_lab.sh" - otrzymuje komunikat: "docker daemon is not available"

Sprawdzam informacje o dockerze i widzę problem w komunikacji.

Problem:

```
vboxuser@Ubuntu:~$ docker info
Client: Docker Engine - Community
Version: 29.7.2
Context: default
Debug Mode: false
Plugins:
buildx: Docker Buildx (Docker Inc.)
Version: v0.36.1
Path: /usr/libexec/docker/cli-plugins/docker-buildx
compose: Docker Compose (Docker Inc.)
Version: v5.5.0
Path: /usr/libexec/docker/cli-plugins/docker-compose

Server:
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

Komunikat zwrotny: "permission denied while trying to connect to the docker API at unix:///var/run/docker.sock - brak uprawnień"

Rozwiązanie problemu:

Uprawnienia do Dockera dla bieżącego użytkownika "vboxuser":

```
vboxuser@Ubuntu:~$ sudo usermod -aG docker vboxuser
```

Dla pewności uruchamiam ponownie Ubuntu 26.04.

Ponowne sprawdzenie "docker info" po zmianach:

```
vboxuser@Ubuntu:~$ docker info
Client: Docker Engine - Community
Version: 29.7.2
Context: default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version: v0.36.1
    Path: /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version: v5.5.0
    Path: /usr/libexec/docker/cli-plugins/docker-compose

Server:
Containers: 1
  Running: 0
  Paused: 0
  Stopped: 1
Images: 1
Server Version: 29.7.2
Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
  /etc/cdi
  /var/run/cdi
Swarm: inactive
Runtimes: io.containerd.runc.v2 runc
Default Runtime: runc
Init Binary: docker-init
  containerd version: aad11006b869517fcd3009450b6f82da282e1a9b
  runc version: v1.4.3-0-gbb14dabe
  init version: de40ad0
Security Options:
  apparmor
  seccomp
   Profile: builtin
  cgroupns
Kernel Version: 7.0.0-30-generic
Operating System: Ubuntu 26.04 LTS
OSType: linux
Architecture: x86_64
CPUs: 3
```

```
Total Memory: 7.232GiB
Name: Ubuntu
ID: 0e5e2832-decd-43dd-bb7b-099d9e2be09d
Docker Root Dir: /var/lib/docker
Debug Mode: false
Experimental: false
Insecure Registries:
  ::1/128
  127.0.0.0/8
Live Restore Enabled: false
Firewall Backend: iptables
  EnableUserlandProxy: true
  UserlandProxyPath: /usr/bin/docker-proxy
```

Inicjuje skrypt uruchamiający “**start_lab.sh**” i potwierdzenie gotowości:

```
vboxuser@Ubuntu: ~/Documents/lab-nginx-http
vboxuser@Ubuntu: ~

vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/start_lab.sh
[+] up 20/20
✓ Image python:3-alpine          Pulled
✓ Image nginx:stable-alpine      Pulled
✓ Network module06_nginx_http_lab_default Created
✓ Container module06_nginx_http_lab-backend-1 Started
✓ Container module06_nginx_http_lab-nginx-1 Started
Waiting for Nginx on http://127.0.0.1:8080/healthz ...

Środowisko startowe działa.
Serwer HTTP:  http://127.0.0.1:8080/
Backend:      wewnętrzna usługa Docker backend:9000

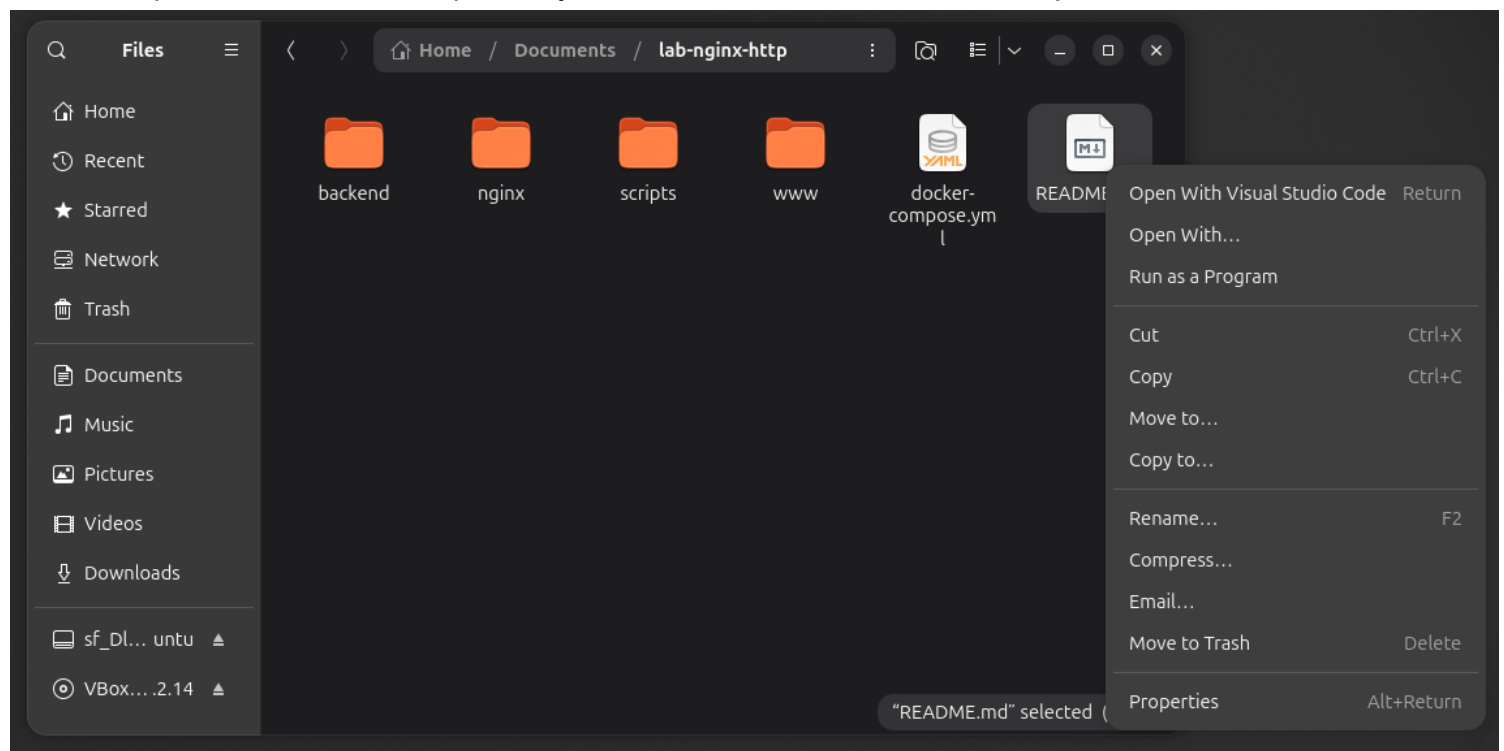
Zadania studenta:
1. Stwórz lokalne CA i certyfikat serwera w nginx/certs/.
2. Dodaj reverse proxy Nginx /api/ do backend:9000.
3. Dodaj blok TLS na porcie 443 i podepnij wygenerowany certyfikat.

Uruchom checker:
./scripts/check_lab.sh

Po zmianie konfiguracji Nginx:
./scripts/reload_nginx.sh
vboxuser@Ubuntu:~/Documents/lab-nginx-http$
```

Teraz mając gotowe narzędzia mogę przejść do rozwiązywania zadania.

Otwieram plik "README.md" za pomocą Visual Studio Code, które wcześniej zainstalowałem



1 # Moduł 06 - lab Nginx HTTP

- 5 - uruchomienie serwera HTTP na Nginx,
- 6 - stworzenie lokalnego CA i certyfikatu serwera,
- 7 - wystawienie reverse proxy z Nginx do usługi backendowej,
- 8 - sprawdzenie wyniku automatycznym checkerem.

10 ## Wymagania

- 12 - Docker z Docker Compose,
- 13 - OpenSSL,
- 14 - curl.

16 ## Uruchomienie

18 ```bash

19 cd content/06/lab-nginx-http

20 ./scripts/start_lab.sh

21 ./scripts/check_lab.sh

22 ```

24 `start\_lab.sh` uruchamia tylko środowisko bazowe. Nie generuje certyfikatów
25 i nie rozwiązuje za studenta zadań z reverse proxy oraz TLS. Student musi uzupełnić:

- 27 - `scripts/generate\_cert.sh` albo stworzyć wymagane pliki certyfikatów ręcznie,
- 28 - `nginx/conf.d/default.conf`,
- 29 - `nginx/includes/lab\_locations.conf`.

31 Po zmianie konfiguracji Nginx:

33 ```bash

34 ./scripts/reload_nginx.sh

35 ```

37 Endpointy:

- 39 - `http://127.0.0.1:8080/` - statyczny serwer HTTP,
- 40 - `https://localhost:8443/` - oczekiwany po wykonaniu zadania z TLS,
- 41 - `https://localhost:8443/api/status` - oczekiwany po wykonaniu zadania z reverse proxy.

43 Jeżeli na porcie `8443` widzisz `400 Bad Request` albo komunikat
44 `The plain HTTP request was sent to HTTPS port`, wysyłasz zwykły HTTP na port
45 TLS. Użyj pełnego adresu `https://localhost:8443/`, a nie
46 `http://localhost:8443/` ani samego `localhost:8443`.

48 Nie uruchamiaj jednocześnie katalogu studenckiego i katalogu z rozwiązaniem,
49 ponieważ oba korzystają z portów `8080` i `8443`.

```
① README.md X $ generate_cert.sh ●
home > vboxuser > Documents > lab-nginx-http > scripts > $ generate_cert.sh
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  usage() {
5  |   cat <<'USAGE'
6  Usage: ./scripts/generate_cert.sh
7
8  To jest skrypt zadaniowy dla studenta. Uzupełnij go tak, aby tworzył:
9  - nginx/certs/student-ca.key
10 - nginx/certs/student-ca.crt
11 - nginx/certs/student-server.key
12 - nginx/certs/student-server.csr
13 - nginx/certs/student-server.crt
14
15 Certyfikat serwera musi być poprawny dla:
16 - localhost
17 - reverse-proxy.lab
18 - 127.0.0.1
19
20 Użyj nginx/certs/ca.cnf oraz nginx/certs/server.cnf jako plików konfiguracyjnych OpenSSL.
```

Edytuje skrypt "generate_cert.sh" o pierwszy krok - "student-ca.key"

Aby najpierw wygenerować klucz prywatny CA: “openssl genrsa -out "\$CERT_DIR/student-ca.key" 4096”

```
File Edit Selection View Go Run Terminal Help
$ generate_cert.sh
home > vboxuser > Documents > lab-nginx-http > scripts > $ generate_cert.sh
35 if [ "${1:-}" = "-h" ] || [ "${1:-}" = "--help" ]; then
36     usage
37     exit 0
38 fi
39
40 if [ "$#" -gt 0 ]; then
41     echo "Ten skrypt zadaniowy nie przyjmuje opcji." >&2
42     usage >&2
43     exit 2
44 fi
45
46 require_cmd openssl
47
48 mkdir -p "$CERT_DIR"
49
50 umask 077
51
52 openssl genrsa -out "$CERT_DIR/student-ca.key" 4096
53
54 cat >&2 <<'TODO'
55 TODO: zaimplementuj generowanie certyfikatow.
56
57 Wymagane pliki wynikowe:
58     nginx/certs/student-ca.key
59     nginx/certs/student-ca.crt
60     nginx/certs/student-server.key
61     nginx/certs/student-server.csr
62     nginx/certs/student-server.crt
63
64 Wskazowki:
65     - Wygeneruj klucz prywatny dla lokalnego CA.
66     - Stworz self-signed certyfikat CA z nginx/certs/ca.cnf.
67     - Wygeneruj klucz prywatny i CSR dla serwera.
68     - Podpisz CSR serwera lokalnym CA.
69     - Przy podpisywaniu uzyj nginx/certs/server.cnf oraz rozszerzen v3_req.
70     - Zweryfikuj wynik poleceniem openssl verify.
71
72 Checker oczekuje dokładnie takich nazw plikow jak powyzej.
73 TODO
74
75 exit 1
```

openssl — uruchamia OpenSSL

genrsa — generuje klucz prywatny RSA

-out — określa, gdzie zapisać wynik

\$CERT_DIR/student-ca.key — ścieżka do klucza

Po wprowadzeniu zmian, uruchamiam zmieniony skrypt:

./scripts/generate_cert.sh

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/generate_cert.sh  
TODO: zaimplementuj generowanie certyfikatow.
```

Wymagane pliki wynikowe:

```
nginx/certs/student-ca.key  
nginx/certs/student-ca.crt  
nginx/certs/student-server.key  
nginx/certs/student-server.csr  
nginx/certs/student-server.crt
```

Wskazowki:

- Wygeneruj klucz prywatny dla lokalnego CA.
- Stworz self-signed certyfikat CA z nginx/certs/ca.cnf.
- Wygeneruj klucz prywatny i CSR dla serwera.
- Podpisz CSR serwera lokalnym CA.
- Przy podpisywaniu uzyj nginx/certs/server.cnf oraz rozszerzen v3_req
- Zweryfikuj wynik poleceniem openssl verify.

Checker oczekuje dokładnie takich nazw plikow jak powyzej.

Sprawdzam, czy plik "student-ca.key" się pojawił:

ls -l nginx/certs/student-ca.key

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ls -l nginx/certs/student-ca.key  
-rw----- 1 vboxuser vboxuser 3272 Aug 24 10:52 nginx/certs/student-ca.key
```

student-ca.key został utworzony po uruchomieniu zmienionego skryptu

właścicielem jest vboxuser

rw----- tylko właściciel może czytać i zapisywać klucz

Sukces. Skoro wiem, że to działa, kontynuuję.

W instrukcji zadania jest wyraźnie napisane, żeby stworzyć self-signed certyfikat CA z nginx/certs/ca.cnf

cat nginx/certs/ca.cnf

Chce podejrzeć parametry ca.cnf:

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ cat nginx/certs/ca.cnf  
[ req ]  
prompt = no  
distinguished_name = dn  
x509_extensions = v3_ca  
  
[ dn ]  
CN = Module 06 Local Lab CA  
O = Cybersecurity Course Local Lab  
OU = Module 06  
  
[ v3_ca ]  
subjectKeyIdentifier = hash  
authorityKeyIdentifier = keyid:always,issuer  
basicConstraints = critical, CA:true  
keyUsage = critical, keyCertSign, cRLSign
```

```
File Edit Selection View Go Run Terminal Help
$ generate_cert.sh
home > vboxuser > Documents > lab-nginx-http > scripts > $ generate_cert.sh
50  umask 077
51
52  openssl genrsa -out "$CERT_DIR/student-ca.key" 4096
53
54  openssl req -x509 \
55  -new \
56  -key "$CERT_DIR/student-ca.key" \
57  -out "$CERT_DIR/student-ca.crt" \
58  -days 365 \
59  -config "$CERT_DIR/ca.cnf"
60
61  cat >&2 <<'TODO'
62  TODO: zaimplementuj generowanie certyfikatow.
63
64  Wymagane pliki wynikowe:
65  nginx/certs/student-ca.key
66  nginx/certs/student-ca.crt
67  nginx/certs/student-server.key
68  nginx/certs/student-server.csr
69  nginx/certs/student-server.crt
70
71  Wskazowki:
72  - Wygeneruj klucz prywatny dla lokalnego CA.
73  - Stworz self-signed certyfikat CA z nginx/certs/ca.cnf.
74  - Wygeneruj klucz prywatny i CSR dla serwera.
75  - Podpisz CSR serwera lokalnym CA.
76  - Przy podpisywaniu uzyj nginx/certs/server.cnf oraz rozszerzen v3_req.
77  - Zweryfikuj wynik poleceniem openssl verify.
78
79  Checker oczekuje dokładnie takich nazw plikow jak powyzej.
80  TODO
81
82  exit 1
```

-config "\$CERT_DIR/ca.cnf"

OpenSSL pobiera tutaj ustawienia potrzebne do stworzenia certyfikatu z pliku ca.cnf

-x509

Oznacza tworzenie certyfikatu X.509 bez self-signed certificate

-req

Wymagane

-new

Utworzenie nowego na podstawie klucza

-key

Użycie naszego prywatnego klucza CA

-out

Zapisywanie certyfikatu do student-ca.crt

-days 365

Ustanawiamy ważność certyfikatu na 365 dni

Po wprowadzeniu nowych zmian, ponownie uruchamiam zmieniony skrypt:

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/generate_cert.sh
TODO: zaimplementuj generowanie certyfikatow.

Wymagane pliki wynikowe:
nginx/certs/student-ca.key
nginx/certs/student-ca.crt
nginx/certs/student-server.key
nginx/certs/student-server.csr
nginx/certs/student-server.crt

Wskazowki:
- Wygeneruj klucz prywatny dla lokalnego CA.
- Stworz self-signed certyfikat CA z nginx/certs/ca.cnf.
- Wygeneruj klucz prywatny i CSR dla serwera.
- Podpisz CSR serwera lokalnym CA.
- Przy podpisywaniu uzyj nginx/certs/server.cnf oraz rozszerzen v3_req
- Zweryfikuj wynik poleceniem openssl verify.

Checker oczekuje dokladnie takich nazw plikow jak powyzej.
```

Sprawdzam, czy zadziało i coś się zmieniło/pojawiło (podobnie jak poprzednio)

ls -l nginx/certs/student-ca.*

* - zastępuje dowolny ciąg znaków

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ls -l nginx/certs/student-ca*
-rw----- 1 vboxuser vboxuser 2029 Aug 24 11:21 nginx/certs/student-ca.crt
-rw----- 1 vboxuser vboxuser 3272 Aug 24 11:21 nginx/certs/student-ca.key
```

Sukces. Mam teraz student-ca.key - czyli, klucz prywatny CA oraz student-ca.crt - czyli, certyfikat CA

Sprawdzam co jest w student-ca.crt:

openssl x509 -in nginx/certs/student-ca.crt -text -noout

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ openssl x509 -in nginx/certs/student-ca.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            09:00:50:16:98:13:7e:6a:40:f6:25:f0:50:34:f7:0c:9c:41:07:cb
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN=Module 06 Local Lab CA, O=Cybersecurity Course Local Lab, OU=Module 06
        Validity
            Not Before: Aug 24 11:21:13 2026 GMT
            Not After : Aug 24 11:21:13 2027 GMT
        Subject: CN=Module 06 Local Lab CA, O=Cybersecurity Course Local Lab, OU=Module 06
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (4096 bit)
            Modulus:
                00:b9:9b:64:ad:c9:f8:ca:2f:39:b0:f3:83:f8:a7:
                04:71:8b:
            Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                C6:4C:9E:3E:D4:55:9A:C4:64:DF:01:7E:C6:68:F5:98:EC:A8:F7:C5
            X509v3 Authority Key Identifier:
                C6:4C:9E:3E:D4:55:9A:C4:64:DF:01:7E:C6:68:F5:98:EC:A8:F7:C5
            X509v3 Basic Constraints: critical
                CA:TRUE
            X509v3 Key Usage: critical
                Certificate Sign, CRL Sign
        Signature Algorithm: sha256WithRSAEncryption
        Signature Value:
            9e:92:ea:80:3b:55:2b:f8:e3:e2:a4:f2:3f:a4:3b:d8:a9:c2:
```

Issuer = Subject, Używa X509v3, CA:TRUE (jest certyfikatem CA)

Póki co wygląda w porządku i mamy wykonane nasze polecenie. Kontynuuj, czas na:

student-server.key

student-server.csr

student-server.crt

Używam polecenia:

openssl genrsa -out "\$CERT_DIR/student-server.key" 2048

```
50 umask 077
51
52 openssl genrsa -out "$CERT_DIR/student-ca.key" 4096
53
54 openssl req -x509 \
55 -new \
56 -key "$CERT_DIR/student-ca.key" \
57 -out "$CERT_DIR/student-ca.crt" \
58 -days 365 \
59 -config "$CERT_DIR/ca.cnf"
60
61 openssl genrsa -out "$CERT_DIR/student-server.key" 2048
62
63 cat >$2 <<'TODO:'
```

Zapisuje zmiany i uruchamiam ponownie skrypt "**generate_cert.sh**"

./scripts/generate_cert.sh

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/generate_cert.sh
TODO: zaimplementuj generowanie certyfikatow.

Wymagane pliki wynikowe:
nginx/certs/student-ca.key
nginx/certs/student-ca.crt
nginx/certs/student-server.key
nginx/certs/student-server.csr
nginx/certs/student-server.crt

Wskazowki:
- Wygeneruj klucz prywatny dla lokalnego CA.
- Stworz self-signed certyfikat CA z nginx/certs/ca.cnf.
- Wygeneruj klucz prywatny i CSR dla serwera.
- Podpisz CSR serwera lokalnym CA.
- Przy podpisywaniu uzyj nginx/certs/server.cnf oraz rozszerzen v3_req.
- Zweryfikuj wynik poleceniem openssl verify.

Checker oczekuje dokładnie takich nazw plikow jak powyzej.
```

Sprawdzam, czy wykonało się:

ls -l nginx/certs/student-server.key

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ls -l nginx/certs/student-server.key
-rw----- 1 vboxuser vboxuser 1704 Aug 24 11:39 nginx/certs/student-server.key
```

cat nginx/certs/server.cnf (Chce podejrzeć parametry **“server.cnf”**)

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ cat nginx/certs/server.cnf
[ req ]
prompt = no
distinguished_name = dn
req_extensions = v3_req

[ dn ]
CN = localhost
O = Cybersecurity Course Local Lab
OU = Module 06

[ v3_req ]
subjectAltName = @alt_names
basicConstraints = critical, CA:false
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth

[ alt_names ]
DNS.1 = localhost
DNS.2 = reverse-proxy.lab
IP.1 = 127.0.0.1
```

Mam teraz student-ca.key - czyli, klucz prywatny CA, student-ca.crt - czyli, certyfikat CA oraz student-server.key - klucz prywatny serwera

Brakuje mi: student-server.csr

.csr będzie zawierał klucz publiczny serwera i informacje dla jakiej tożsamości chcemy certyfikat.

Do tego celu potrzebny jest ciąg komend:

```
openssl req -new \  
-key "$CERT_DIR/student-server.key" \  
-out "$CERT_DIR/student-server.csr" \  
-config "$CERT_DIR/server.cnf
```

-new

Nowe żądanie CSR

```
-key "$CERT_DIR/student-server.key"
```

Używam tego konkretnego klucza prywatnego serwera

```
-out "$CERT_DIR/student-server.csr"
```

Zapisuje wynik jako student-server-csr

```
-config "$CERTDIR/server.cnf
```

Analogicznie jak poprzednio wskazuje na użycie konfiguracji serwera z server.cnf

server.cnf w swoich parametrach **req_extensions = v3_req** powoduje, że jest on używany przy tworzeniu CSR

```

50 umask 077
51
52 openssl genrsa -out "$CERT_DIR/student-ca.key" 4096
53
54 openssl req -x509 \
55 -new \
56 -key "$CERT_DIR/student-ca.key" \
57 -out "$CERT_DIR/student-ca.crt" \
58 -days 365 \
59 -config "$CERT_DIR/ca.cnf"
60
61 openssl genrsa -out "$CERT_DIR/student-server.key" 2048
62
63 openssl req -new \
64 -key "$CERT_DIR/student-server.key" \
65 -out "$CERT_DIR/student-server.csr" \
66 -config "$CERT_DIR/server.cnf"
67
68 cat >f3 <<[TODO]

```

Po wprowadzeniu nowych zmian, zapisaniu ich, uruchamiam ponownie skrypt:

./scripts/generate_cert.sh

```

vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/generate_cert.sh
TODO: zaimplementuj generowanie certyfikatow.

Wymagane pliki wynikowe:
nginx/certs/student-ca.key
nginx/certs/student-ca.crt
nginx/certs/student-server.key
nginx/certs/student-server.csr
nginx/certs/student-server.crt

Wskazowki:
- Wygeneruj klucz prywatny dla lokalnego CA.
- Stworz self-signed certyfikat CA z nginx/certs/ca.cnf.
- Wygeneruj klucz prywatny i CSR dla serwera.
- Podpisz CSR serwera lokalnym CA.
- Przy podpisywaniu uzyj nginx/certs/server.cnf oraz rozszerzen v3_req.
- Zweryfikuj wynik poleceniem openssl verify.

Checker oczekuje dokladnie takich nazw plikow jak powyzej.

```

Sprawdzam, czy istnieje teraz plik "student-server.csr"

ls -l nginx/certs/student-server.csr

```

vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ls -l nginx/certs/student-server.csr
-rw----- 1 vboxuser vboxuser 1131 Aug 24 12:05 nginx/certs/student-server.csr

```

Istnieje, więc chce teraz jego zawartość podejrzeć w formie tekstowej:

openssl req -in nginx/certs/student-server.csr -text -noout

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ openssl req -in nginx/certs/student-server.csr -text -noout
Certificate Request:
Data:
  Version: 1 (0x0)
  Subject: CN=localhost, O=Cybersecurity Course Local Lab, OU=Module 06
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:a7:fa:57:15:b8:a6:34:26:fa:1b:5f:5e:5e:b6:
        cd:22:66:a8:15:51:33:08:11:c8:d0:a0:f5:38:c7:
        1f:1c:c7:f4:60:61:ee:24:04:51:8a:43:d4:52:44:
        06:13:9e:67:81:14:1a:93:43:e5:58:2c:7a:bd:c6:
        d7:7b:dc:48:45:8f:26:4d:71:9a:50:23:7b:d9:d6:
        17:6a:a9:46:5d:e0:d2:88:fe:71:59:b7:09:6a:76:
        96:b1:26:05:5e:40:69:34:72:ce:a2:0a:fd:75:ee:
        70:3b:60:71:92:23:08:6d:04:42:7d:b3:50:4b:b5:
        d1:5e:ca:e3:b0:d9:c3:4b:70:76:d7:0e:83:eb:41:
        25:6a:c1:d1:05:a4:82:e8:71:56:de:a3:cb:87:5e:
        ea:23:91:c7:03:ba:ee:73:88:7a:41:69:fe:e6:e4:
        91:30:e1:d7:d0:e2:b7:bc:5f:30:21:fa:d0:f5:9d:
        02:14:1c:d8:be:92:0f:bc:1e:ae:ef:c2:ea:da:eb:
        24:c3:9f:04:5f:e9:82:8d:ca:8b:d0:c8:b3:14:0e:
        0f:4a:9b:c0:b1:83:37:81:68:4a:c6:ba:b1:e9:64:
        eb:ee:f3:ca:7b:34:c5:ad:ad:b1:91:cf:ec:89:19:
        4e:e9:c2:15:b9:c1:50:db:67:07:3c:cb:ed:e1:a2:
        b1:bf
      Exponent: 65537 (0x10001)
  Attributes:
    Requested Extensions:
      X509v3 Subject Alternative Name:
        DNS:localhost, DNS:reverse-proxy.lab, IP Address:127.0.0.1
      X509v3 Basic Constraints: critical
        CA:FALSE
      X509v3 Key Usage: critical
        Digital Signature, Key Encipherment
      X509v3 Extended Key Usage:
        TLS Web Server Authentication
  Signature Algorithm: sha256WithRSAEncryption
  Signature Value:
    51:c0:e1:eb:04:7d:d0:2c:fc:e4:d6:d0:6a:0d:c0:06:d4:b1:
```

"Subject: CN=localhost, O=Cybersecurity Course Local Lab, OU=Module 06"

"Public-Key: (2048 bit)"

SAN (Subject Alternative Name):

X509v3 Subject Alternative Name:

"DNS:localhost, DNS:reverse-proxy.lab, IP Address:127.0.0.1"

Czyli ma to, czego wymaga zadanie:

localhost, reverse-proxy.lab, 127.0.0.1

Basic Constraints:

"X509v3 Basic Constraints: critical

CA:FALSE"

Key Usage:

"Digital Signature, Key Encipherment" z server.cnf

Extended Key Usage:

"TLS Web Server Authentication"

Na tym etapie mam teraz student-ca.key - czyli, klucz prywatny CA, student-ca.crt - czyli, certyfikat CA, student-server.key - klucz prywatny serwera, student-server.csr - żądanie podpisania certyfikatu

Brakuje mi: student-server.crt

Do skryptu "generate_cert.sh" dodaje:

```
openssl x509 -req \  
-in "$CERT_DIR/student-server.csr" \  
-CA "$CERT_DIR/student-ca.crt" \  
-CAkey "$CERT_DIR/student-ca.key" \  
-CAcreateserial \  
-out "$CERT_DIR/student-server.crt" \  
-days 365 \  
-extfile "$CERT_DIR/server.cnf" \  
-extensions v3_req
```

-req

Komunikuje OpenSSL, że wejściem jest CSR

-in "\$CERT_DIR/student-server.csr"

To żądanie, które przed chwilą sprawdzałem

-CA "\$CERT_DIR/student-ca.crt"

Tutaj podaje certyfikat CA, czyli "student-ca.crt" jest certyfikatem wystawcy

-CAkey "\$CERT_DIR/student-ca.key"

Tutaj podaje prywatny klucz CA, za jego pomocą OpenSSL stworzy podpis certyfikatu

-CAcreateserial

Ta opcja powoduje utworzenie pliku pomocniczego z numerem seryjnym "student-ca.srl", nie jest to ani certyfikat, ani klucz

-out "\$CERT_DIR/student-server.crt"

Wskazuje tutaj wynik do "student-server.crt", czyli właśnie certyfikat serwera, który jest niezbędny dla NGINX

-days 365

Określam ważność certyfikatu, tutaj 365 dni

-extfile "\$CERT_DIR/server.cnf" -extensions v3_req

To jest ważne dlatego, że CSR już zawierał te informacje, ale podczas wystawiania finalnego certyfikatu chce zastosować rozszerzenia v3_req. Instrukcja zadania wprost wymaga użycia server.cnf i rozszerzeń v3_req

W "server.cnf" mam przecież:

"[v3_req]

subjectAltName = @alt_names

basicConstraints = critical, CA:false

keyUsage = critical, digitalSignature, keyEncipherment

extendedKeyUsage = serverAuth"

"[alt_names]

DNS.1 = localhost

DNS.2 = reverse-proxy.lab

IP.1 = 127.0.0.1"

```

50 umask 077
51
52 openssl genrsa -out "$CERT_DIR/student-ca.key" 4096
53
54 openssl req -x509 \
55 -new \
56 -key "$CERT_DIR/student-ca.key" \
57 -out "$CERT_DIR/student-ca.crt" \
58 -days 365 \
59 -config "$CERT_DIR/ca.cnf"
60
61 openssl genrsa -out "$CERT_DIR/student-server.key" 2048
62
63 openssl req -new \
64 -key "$CERT_DIR/student-server.key" \
65 -out "$CERT_DIR/student-server.csr" \
66 -config "$CERT_DIR/server.cnf"
67
68 openssl x509 -req \
69 -in "$CERT_DIR/student-server.csr" \
70 -CA "$CERT_DIR/student-ca.crt" \
71 -CAkey "$CERT_DIR/student-ca.key" \
72 -CAcreateserial \
73 -out "$CERT_DIR/student-server.crt" \
74 -days 365 \
75 -extfile "$CERT_DIR/server.cnf" \
76 -extensions v3_req
77
78

```

Zapisuje zmienioną treść i uruchamiam skrypt:

./scripts/generate_cert.sh

```

vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/generate_cert.sh
Certificate request self-signature ok
subject=CN=localhost, O=Cybersecurity Course Local Lab, OU=Module 06
TODO: zaimplementuj generowanie certyfikatow.

Wymagane pliki wynikowe:
  nginx/certs/student-ca.key
  nginx/certs/student-ca.crt
  nginx/certs/student-server.key
  nginx/certs/student-server.csr
  nginx/certs/student-server.crt

Wskazowki:
- Wygeneruj klucz prywatny dla lokalnego CA.
- Stworz self-signed certyfikat CA z nginx/certs/ca.cnf.
- Wygeneruj klucz prywatny i CSR dla serwera.
- Podpisz CSR serwera lokalnym CA.
- Przy podpisywaniu uzyj nginx/certs/server.cnf oraz rozszerzen v3_req.
- Zweryfikuj wynik poleceniem openssl verify.

Checker oczekuje dokladnie takich nazw plikow jak powyzej.

```

Sprawdzam liste certyfikatow:

ls -la nginx/certs/

```

vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ls -l nginx/certs/
total 32
-rwxrwx--- 1 vboxuser vboxuser 317 Aug 20 17:54 ca.cnf
-rwxrwx--- 1 vboxuser vboxuser 380 Aug 20 17:54 server.cnf
-rw----- 1 vboxuser vboxuser 2029 Aug 24 12:42 student-ca.crt
-rw----- 1 vboxuser vboxuser 3272 Aug 24 12:42 student-ca.key
-rw----- 1 vboxuser vboxuser 41 Aug 24 12:42 student-ca.srl
-rw----- 1 vboxuser vboxuser 1757 Aug 24 12:42 student-server.crt
-rw----- 1 vboxuser vboxuser 1131 Aug 24 12:42 student-server.csr
-rw----- 1 vboxuser vboxuser 1704 Aug 24 12:42 student-server.key

```

Posiadam juz komplet certyfikatow

Chcę potwierdzić jeszcze, że student-ca.crt podpisuje student-server.crt

openssl verify -CAfile nginx/certs/student-ca.crt nginx/certs/student-server.crt

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ openssl verify -CAfile nginx/certs/student-ca.crt nginx/certs/student-server.crt
nginx/certs/student-server.crt: OK
```

-CAfile student-ca.crt

Wskazuje certyfikat wystawcy, a ostatni argument wskazuje certyfikat serwera, który sprawdzam

Przechodzę teraz do zdefiniowania backendu dla /api/ w pliku konfiguracyjnym: **"default.conf"**

Muszę wskazać NGINX-owi, gdzie znajduje się aplikacja backendowa, do której ma on przekazywać żądania.

W Dockerze backend, jest pod nazwą kontenera "backend". Mam pamiętać również o porcie 9000.

```
home > vboxuser > Documents > lab-nginx-http > nginx > conf.d > default.conf
1  upstream backend_app {
2      server backend:9000;
3  }
4
5  server {
6      listen 80 default_server;
7      server_name localhost reverse-proxy.lab;
8
9      server_tokens off;
10     client_max_body_size 1m;
11     client_header_timeout 10s;
12     client_body_timeout 10s;
13     send_timeout 30s;
14
15     include /etc/nginx/includes/lab_locations.conf;
16 }
17
18 # TODO 1:
19 # Dodaj upstream o nazwie backend_app, który wskazuje na backend:9000.
20
21 # TODO 2:
22 # Dodaj blok HTTPS server na porcie 443.
23 # Powinien używać:
24 #     ssl_certificate /etc/nginx/certs/student-server.crt;
25 #     ssl_certificate_key /etc/nginx/certs/student-server.key;
26 # W bloku HTTPS też dołącz /etc/nginx/includes/lab_locations.conf.
27
```

Uruchamiam środowisko testowe skryptem:

./scripts/start_lab.sh

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/start_lab.sh
[+] up 2/2
✓ Container module06_nginx_http_lab-backend-1 Started
✓ Container module06_nginx_http_lab-nginx-1 Started
Waiting for Nginx on http://127.0.0.1:8080/healthz ...
```

Środowisko startowe działa.

```
Serwer HTTP:  http://127.0.0.1:8080/
Backend:      wewnętrzna usługa Docker backend:9000
```

Zadania studenta:

1. Stwórz lokalne CA i certyfikat serwera w nginx/certs/.
2. Dodaj reverse proxy Nginx /api/ do backend:9000.
3. Dodaj blok TLS na porcie 443 i podepnij wygenerowany certyfikat.

Uruchom checker:

```
./scripts/check_lab.sh
```

Po zmianie konfiguracji Nginx:

```
./scripts/reload_nginx.sh
```

Czas na skonfigurowanie HTTPS (nasłuch portu 443 z ssl)

Pod pierwszym blokiem server 80, dodaje drugi blok server dla 443 z ssl:

```
"server {  
  
    listen 443 ssl;  
  
    server_name localhost reverse-proxy.lab;  
  
  
    ssl_certificate /etc/nginx/certs/student-server.crt;  
  
    ssl_certificate_key /etc/nginx/certs/student-server.key;  
  
  
  
    server_tokens off;  
  
    client_max_body_size 1m;  
  
    client_header_timeout 10s;  
  
    client_body_timeout 10s;  
  
    send_timeout 30s;  
  
  
  
    include /etc/nginx/includes/lab_locations.conf;  
  
}"
```

"listen 443 ssl;" - 443 to numer portu dla HTTPS, dodatkowe ssl mówi NGINX-owi, żeby na tym porcie oczekiwał połączenia z TLS i HTTP, a nie zwykłego HTTP

"ssl_certificate /etc/nginx/certs/student-server.crt;

ssl_certificate_key /etc/nginx/certs/student-server.key;" - pierwsza linia wskazuje certyfikat serwera,

druga linia wskazuje jego klucz prywatny

```
home > vboxuser > Documents > lab-nginx-http > nginx > conf.d > default.conf
1  upstream backend_app {
2      server backend:9000;
3  }
4
5  server {
6      listen 80 default_server;
7      server_name localhost reverse-proxy.lab;
8
9      server_tokens off;
10     client_max_body_size 1m;
11     client_header_timeout 10s;
12     client_body_timeout 10s;
13     send_timeout 30s;
14
15     include /etc/nginx/includes/lab_locations.conf;
16 }
17
18 server {
19     listen 443 ssl;
20     server_name localhost reverse-proxy.lab;
21
22     ssl_certificate /etc/nginx/certs/student-server.crt;
23     ssl_certificate_key /etc/nginx/certs/student-server.key;
24
25     server_tokens off;
26     client_max_body_size 1m;
27     client_header_timeout 10s;
28     client_body_timeout 10s;
29     send_timeout 30s;
30
31     include /etc/nginx/includes/lab_locations.conf;
32 }
```

Zapisuje zmiany w pliku "**default.conf**" i wykonuje skrypt "**reload_nginx.sh**" (readme zaleca takie działanie po zmianie konfiguracji NGINX-a):

```
home > vboxuser > Documents > lab-nginx-http > README.md > abc # Moduł 06 - lab Nginx HTTP
1  # Moduł 06 - lab Nginx HTTP
2
3  Lab dla studentów obejmuje:
4
5  - uruchomienie serwera HTTP na Nginx,
6  - stworzenie lokalnego CA i certyfikatu serwera,
7  - wystawienie reverse proxy z Nginx do usługi backendowej,|
8  - sprawdzenie wyniku automatycznym checkerem.
9
10 ## Wymagania
11
12 - Docker z Docker Compose,
13 - OpenSSL,
14 - curl.
15
16 ## Uruchomienie
17
18 ```bash
19 cd content/06/lab-nginx-http
20 ./scripts/start_lab.sh
21 ./scripts/check_lab.sh
22 ```
23
24 `start_lab.sh` uruchamia tylko środowisko bazowe. Nie generuje certyfikatów
25 i nie rozwiązuje za studenta zadań z reverse proxy oraz TLS. Student musi uzupełnić:
26
27 - `scripts/generate_cert.sh` albo stworzyć wymagane pliki certyfikatów ręcznie,
28 - `nginx/conf.d/default.conf`,
29 - `nginx/includes/lab_locations.conf`.
30
31 Po zmianie konfiguracji Nginx:
32
33 ```bash
34 ./scripts/reload_nginx.sh
35 ```
```

reload_nginx.sh

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/reload_nginx.sh
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/08/24 13:47:31 [notice] 35#35: signal process started
```

Przetestuje połączenie HTTPS za pomocą

curl -vk <https://localhost:8443/>

Tutaj -v pokaże mi szczegóły połączenia TLS, a -k pozwoli na połączenie mimo tego, że mój lokalny CA nie znajduje się w systemowym magazynie zaufanych CA

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ curl -vk https://localhost:8443/
* Host localhost:8443 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8443...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* SSL Trust: peer verification disabled
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768 / RSASSA-PSS
* ALPN: server accepted http/1.1
* Server certificate:
*   subject: CN=localhost; O=Cybersecurity Course Local Lab; OU=Module 06
*   start date: Aug 24 12:42:27 2026 GMT
*   expire date: Aug 24 12:42:27 2027 GMT
*   issuer: CN=Module 06 Local Lab CA; O=Cybersecurity Course Local Lab; OU=Module 06
*   Certificate level 0: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
* SSL certificate verification failed, continuing anyway!
* Established connection to localhost (::1 port 8443) from ::1 port 34400
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:8443
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Mon, 24 Aug 2026 13:48:31 GMT
< Content-Type: text/html
< Content-Length: 146
< Connection: keep-alive
< X-Content-Type-Options: nosniff
< Referrer-Policy: strict-origin-when-cross-origin
< X-Lab-Server: nginx-static-http-server
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
< X-Lab-Server: nginx-static-http-server
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
* Connection #0 to host localhost:8443 left intact
```

Wniosek jest taki, że HTTPS działa poprawnie:

“SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384”

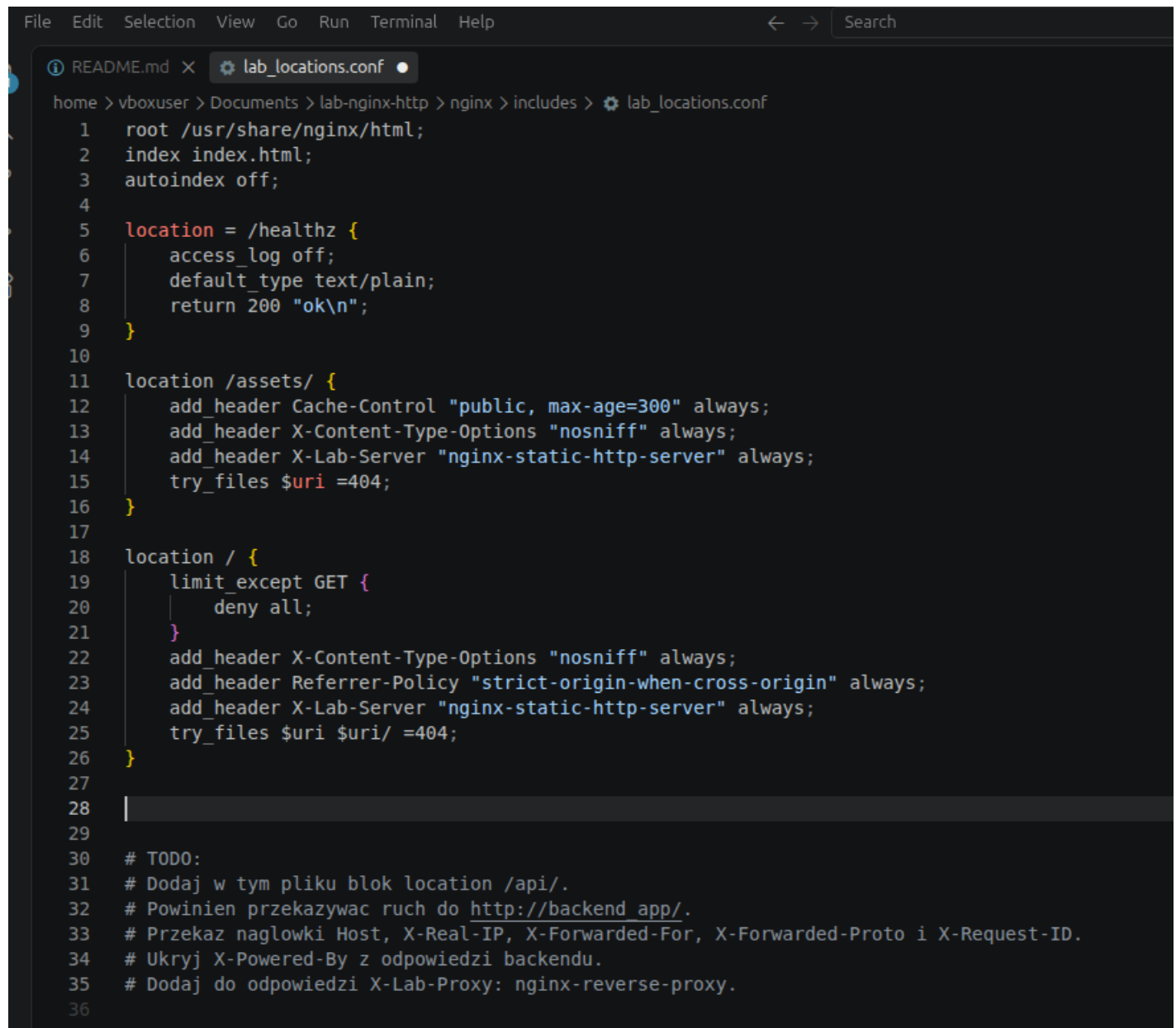
“subject: CN=localhost

issuer: CN=Module 06 Local Lab CA”

NGINX faktycznie korzysta z student-server.crt i połączenie wykorzystuje TLS w wersji 1.3

Mamy również działający wymagany z readme endpoint: <https://localhost:8443/> (w poleceniu)

Za pomocą Visual Studio Code otwieram plik: "nginx/includes/lab_locations.conf"



```
home > vboxuser > Documents > lab-nginx-http > nginx > includes > lab_locations.conf
1  root /usr/share/nginx/html;
2  index index.html;
3  autoindex off;
4
5  location = /healthz {
6      access_log off;
7      default_type text/plain;
8      return 200 "ok\n";
9  }
10
11 location /assets/ {
12     add_header Cache-Control "public, max-age=300" always;
13     add_header X-Content-Type-Options "nosniff" always;
14     add_header X-Lab-Server "nginx-static-http-server" always;
15     try_files $uri =404;
16 }
17
18 location / {
19     limit_except GET {
20         deny all;
21     }
22     add_header X-Content-Type-Options "nosniff" always;
23     add_header Referrer-Policy "strict-origin-when-cross-origin" always;
24     add_header X-Lab-Server "nginx-static-http-server" always;
25     try_files $uri $uri/ =404;
26 }
27
28 |
29
30 # TODO:
31 # Dodaj w tym pliku blok location /api/.
32 # Powinien przekazywać ruch do http://backend app/.
33 # Przekaz nagłówki Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto i X-Request-ID.
34 # Ukryj X-Powered-By z odpowiedzi backendu.
35 # Dodaj do odpowiedzi X-Lab-Proxy: nginx-reverse-proxy.
36
```

Na końcu pliku ale jeszcze przed komentarzami dodaje nowy blok:

“location /api/ {

proxy_pass http://backend_app/;

proxy_set_header Host \$host;

proxy_set_header X-Real-IP \$remote_addr;

proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;

proxy_set_header X-Forwarded-Proto \$scheme;

proxy_set_header X-Request-ID \$request_id;

proxy_hide_header X-Powered-By;

add_header X-Lab-Proxy "nginx-reverse-proxy" always;

```
}"
```

```
"proxy_pass http://backend\_app/;"
```

Nawiązuje do wcześniej dopisywanego:

```
"upstream backend_app {
```

```
    server backend:9000;
```

```
}"
```

i tworzy właściwą kolejność i przepływ Nginx :443 (location /api/) -> backend_app -> backend:9000

Wyjaśnienie przekazywanych nagłówków:

```
"proxy_set_header X-Real-IP $remote_addr;"
```

Przekazuje backendowi IP klienta

```
"proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;"
```

Buduje informację o adresach klientów/proxy

```
"proxy_set_header X-Forwarded-Proto $scheme;"
```

Informuje backend, czy żądanie przyszło przez http czy https, w naszym przypadku https

```

home > vboxuser > Documents > lab-nginx-http > nginx > includes > ⚙ lab_locations.conf
1  root /usr/share/nginx/html;
2  index index.html;
3  autoindex off;
4
5  location = /healthz {
6      access_log off;
7      default_type text/plain;
8      return 200 "ok\n";
9  }
10
11 location /assets/ {
12     add_header Cache-Control "public, max-age=300" always;
13     add_header X-Content-Type-Options "nosniff" always;
14     add_header X-Lab-Server "nginx-static-http-server" always;
15     try_files $uri =404;
16 }
17
18 location / {
19     limit_except GET {
20         deny all;
21     }
22     add_header X-Content-Type-Options "nosniff" always;
23     add_header Referrer-Policy "strict-origin-when-cross-origin" always;
24     add_header X-Lab-Server "nginx-static-http-server" always;
25     try_files $uri $uri/ =404;
26 }
27
28 location /api/ {
29     proxy_pass http://backend_app/;
30
31     proxy_set_header Host $host;
32     proxy_set_header X-Real-IP $remote_addr;
33     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
34     proxy_set_header X-Forwarded-Proto $scheme;
35     proxy_set_header X-Request-ID $request_id;
36
37     proxy_hide_header X-Powered-By;
38     add_header X-Lab-Proxy "nginx-reverse-proxy" always;
39 }
40
41 # TODO:
42 # Dodaj w tym pliku blok location /api/.
43 # Powinien przekazywać ruch do http://backend_app/.
44 # Przekaz nagłówki Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto i X-Request-ID.
45 # Ukryj X-Powered-By z odpowiedzi backendu.
46 # Dodaj do odpowiedzi X-Lab-Proxy: nginx-reverse-proxy.

```

Zapisuje zmiany w pliku **"lab_locations.conf"** i wykonuje skrypt **"reload_nginx.sh"** (readme zaleca takie działanie po zmianie konfiguracji NGINX-a):

```

vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/reload_nginx.sh
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/08/24 14:17:19 [notice] 50#50: signal process started

```

Testuje teraz wymagany przez README endpoint:

curl -vk <https://localhost:8443/api/status>

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ curl -vk https://localhost:8443/api/status
* Host localhost:8443 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8443...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* SSL Trust: peer verification disabled
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768 / RSASSA-PSS
* ALPN: server accepted http/1.1
* Server certificate:
* subject: CN=localhost; O=Cybersecurity Course Local Lab; OU=Module 06
* start date: Aug 24 12:42:27 2026 GMT
* expire date: Aug 24 12:42:27 2027 GMT
* issuer: CN=Module 06 Local Lab CA; O=Cybersecurity Course Local Lab; OU=Module 06
* Certificate level 0: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
* SSL certificate verification failed, continuing anyway!
* Established connection to localhost (::1 port 8443) from ::1 port 49418
* using HTTP/1.x
> GET /api/status HTTP/1.1
> Host: localhost:8443
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
< HTTP/1.1 200 OK
< Server: nginx
< Date: Mon, 24 Aug 2026 14:18:38 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 315
< Connection: keep-alive
< Cache-Control: no-store
< X-Backend-ID: backend-01
< X-Lab-Proxy: nginx-reverse-proxy
<
{
  "backend_id": "backend-01",
  "client_address": "172.18.0.3",
  "host": "localhost",
  "method": "GET",
  "path": "/status",
  "service": "backend-http-server",
  "x_forwarded_for": "172.18.0.1",
  "x_forwarded_proto": "https",
  "x_real_ip": "172.18.0.1",
  "x_request_id": "eff9420fb697d4fccf174a2dcbcd24d6"
}
* Connection #0 to host localhost:8443 left intact
```

"<HTTPS/1.1 200 OK"

Oznacza to, że działa poprawnie

"< X-Backend-ID: backend-01"

< X-Lab-Proxy: nginx-reverse-proxy"

X-Lab-Proxy jest nagłówkiem, z lab_locations.conf. To potwierdza, że odpowiedź przeszła przez Nginxa

Mam też odpowiedź z backendu:

"{"

```
"backend_id": "backend-01",
"client_address": "172.18.0.3",
"host": "localhost",
"method": "GET",
"path": "/status",
"service": "backend-http-server",
"x_forwarded_for": "172.18.0.1",
"x_forwarded_proto": "https",
"x_real_ip": "172.18.0.1",
"x_request_id": "...",
}
```

"path" podaje /status, a wysłałem **"/api/status "**, więc zadziałał wpis:

“location /api/ {

proxy_pass http://backend_app/;

}"

Prefiks /api/ został zastąpiony /, więc backend dostał /status.

"x_forwarded_proto": "https"

Pochodzi z wpisu:

“proxy_set_header X-Forwarded-Proto \$scheme;”

Backend wie więc, że klient pierwotnie połączył się z Nginxem przez **HTTPS**, mimo że sam Nginx komunikuje się z backendem przez HTTP

Mapa przepływu:

curl

|

| HTTPS

| <https://localhost:8443/api/status>

▼

VirtualBox / Ubuntu

|

| port 8443 → 443

▼

Nginx

|

| TLS + student-server.crt

|

| location /api/

|

| proxy_pass



backend_app

|

| HTTP



backend:9000

|

| /status



backend-01

Na koniec uruchamiam ostatni skrypt sprawdzający:

./scripts/check_lab.sh

```
vboxuser@Ubuntu:~/Documents/lab-nginx-http$ ./scripts/check_lab.sh
[PASS] Command available: docker
[PASS] Command available: openssl
[PASS] Command available: curl
[PASS] Docker daemon is available
[PASS] Certificate files exist
[PASS] Server certificate verifies against local CA
[PASS] Server certificate contains DNS SAN: localhost
[PASS] Server certificate contains DNS SAN: reverse-proxy.lab
[PASS] Server certificate contains IP SAN: 127.0.0.1
[PASS] Nginx container is running
[PASS] Backend container is running
[PASS] Nginx configuration test passes
[PASS] HTTP health endpoint responds
[FAIL] Nginx does not serve the expected static HTTP page
[PASS] Static asset has expected Cache-Control header
[PASS] Backend is not directly exposed on host port 9000
[PASS] HTTPS endpoint validates with the local CA
[PASS] Reverse proxy reaches the backend service
[PASS] Reverse proxy forwards X-Forwarded-Proto
[PASS] Reverse proxy adds X-Lab-Proxy header
[PASS] Upstream X-Powered-By header is hidden by Nginx

Summary: 20 passed, 1 failed, 0 warnings
```

“1 failed”: “[FAIL] Nginx does not serve the expected static http page”